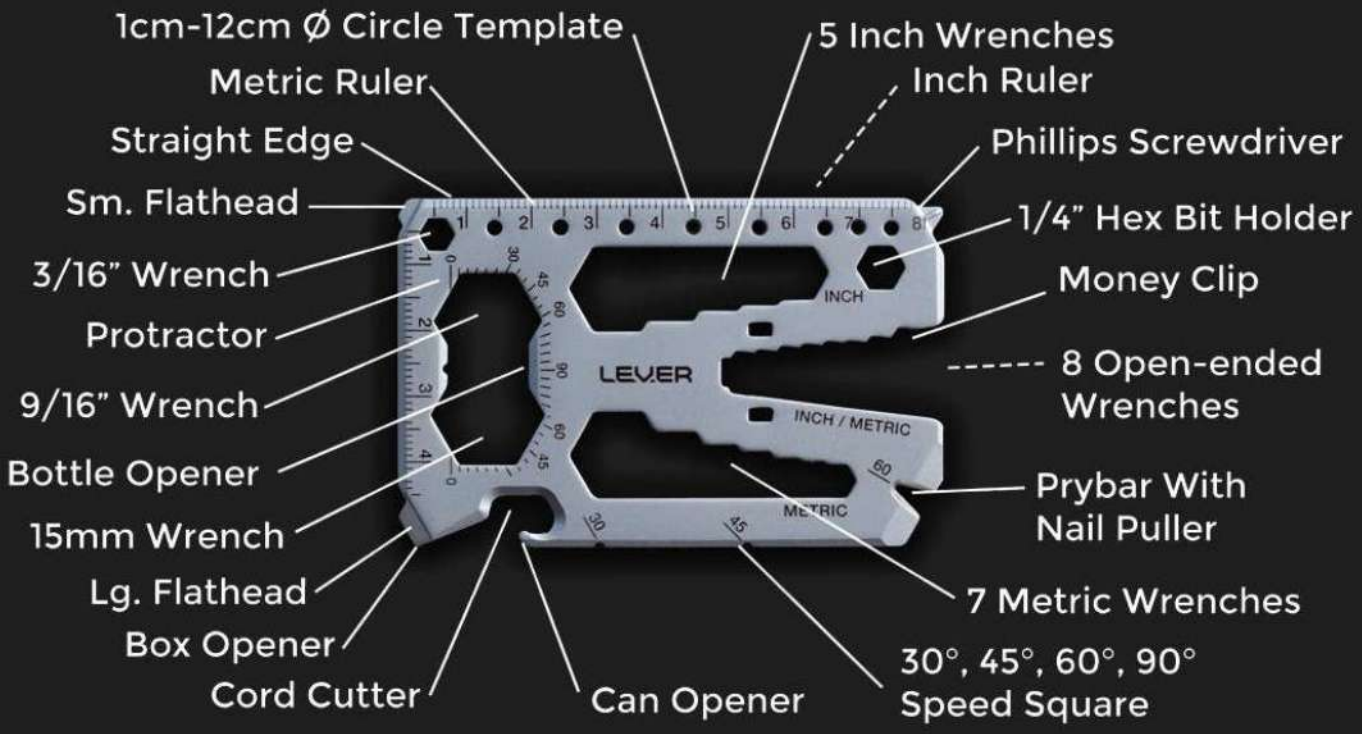
Boken du håller i ska hjälpa dig hitta din unika potential. För att göra det ska den ge dig ett inre landskap på vad du kan göra för att se helheten innan du gör något som är viktigt.

Daniel Åke Olov Jonsson

Road 30

Daniel Åke Olov Jonsson



2. Epistemology

3. Puzzle

4. Mechanics

5. Motives

6. Needs

7. Psychiatry

8. Artificial intellige

9. Music

10. Entreprenurship

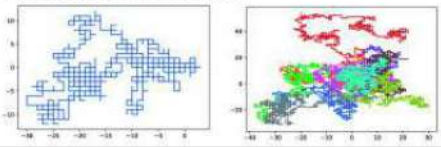
11. Swedish laws

12. Exercises

A 1. Load the caffeine molecule from SMILES
2. Draw it

B
```
def gauss(smiles):
    """This function prepares a Gaussian input file from the
given smiles using rdkit to prepare initial coordintes by
    1. Converting SMILES to molecule
    2. Preparing 3D coordinates
    3. Writing out Gaussian input file"""
    # Convert SMILES to molecule
    mol = Chem.MolFromSmiles(smiles)
    # Generate 3D coordinates
    mol = Chem.AddHs(mol)
    AllChem.EmbedMolecule(mol)
    AllChem.MMFFOptimizeMolecule(mol)
    # Write out Gaussian input file
    file_name = smiles + '.com'
    with open(file_name, 'w') as f:
        f.write('%chk=' + smiles + '.chk\n')
        f.write('%nprocshared=4\n')
        f.write('%mem=4GB\n')
        f.write('#p opt b3lyp/6-31g(d) empiricaldisper-
sion=gd3\n\n')
        f.write(smiles + '\n\n')
        f.write('0 1\n')
        for atom in mol.GetAtoms():
            pos = mol.GetConformer().GetAtomPosition(a-
tom.GetIdx())
            f.write('{:<4} {:>10.5f} {:>10.5f} {:>10.5f}\n'.format(a-
tom.GetSymbol(), pos.x, pos.y, pos.z))
        f.write('\n\n')
    return file_name
```

C - This script generates the behavior of a 2D random walk and plots trajectories
- Rewrite this into a function



D
```
# this function downloads a pdb file from the web and
saves it to the current directory
def download_pdb(pdb_id):
    pdb_file = pdb_id + ".pdb"
    url = "https://files.rcsb.org/download/" + pdb_file
    response = urllib.request.urlopen(url)
    data = response.read()
    pdb_file = open(pdb_file, 'wb')
    pdb_file.write(data)
    pdb_file.close()
    return pdb_file

# download the structure of 1lys
pdb_id = "1lys"
pdb_file = download_pdb(pdb_id)

# now compute the radius of gyration of 1lys with mdtraj
# load the pdb file into mdtraj
traj = md.load(pdb_file.name)
# compute the radius of gyration
rg = md.compute_rg(traj)
print(rg)
```

AWS feels like toolbox. If you want to do something, there's probably something available in there to help do it. However, there's very little (or, in a couple cases, none at all) guidance on how those tools should actually be used.

Many things have sensible defaults, but there's often very little explanations on why you might enable/disable some of those defaults. Or the best ways to integrate different services (for example, S3 supports notifications via Lambda, SNS, and SQS, but offers no input on why you might pick one over the others).

The reason seems simple enough: you're not expected to use the tools if you don't already know how to use them. You're not expected to pop open ElastiCache and easily be able to use it. You're expected to think "I wish I could use Redis for this... how can I do that with AWS?"

And on a completely unrelated note: the Console should absolutely have a way to export resources as CloudFormation/Terraform templates. Build it in the console in an exploratory account, test it out, export the template, and use the template to recreate it in the dev/prod accounts. This should be something every single service supports, would integrate great with Organizations/Control Tower, would drive experimentation, and would help encourage better prod environments for people.

Sentence: In brief, fluorescent dyes dissolved in tetrahydrofuran (THF) were added to particles stabilized with Pluronic F108 (MilliporeSigma) to a final concentration of 30% v/v THF, then diluted by a factor of five before washing the particles via multiple sedimentation and resuspension cycles to set them in pure water.

Chemical Entities: **Pluronic F108, tetrahydrofuran, THF**

70% open rate, 5% reply rate, 3% bookings, 20% closing rate, 100% refund guarantee.

5 personliga inbjudningar kvar för den här månaden.

Me: Tech Sales. I do bookings so far because I don't want to hire a Product Manager after selling my SaaS to D2D. Ask me anything on mailbox. Pride before the Fall. 🦊

Brageon

Leads agency. Market research. Specialist in fintech and telecom.

Marknadsföringstjänster • Stockholm, Stockholm
1 följare • 0-1 anställda

I consent to be contacted for marketing purposes by Hotjar's parent company, Contentsquare SAS (and its affiliates). Read more

Procedural: 1) horizontal puzzles, 2) vertical parameters, 3) congruent correlations, 4) explicit prediction inside vehicles. Ranking matrices based on similarity to correlations. COVID reduced CO2. Not by regulations but instead of distance jobs. Correlation (government) is not causation (less CO2) by a stronger link to consulting. [0/2–1/2, 2*0/1+2]. Flanker task is used to (identify heuristic) to (transfer parameters) in different examples unlike Wason selection task [1*(2+3)+(2+3)/3].

♉ Grief, bewildered. ♒ Facts to motivate. Type shovel to crops. Be hated by foolish king from altruism. ♏ Lifting up enemy, bewildered, new idol, tower, more rows. ♌ Self-sufficient. ♑ Free people from fruitless paths. ♈ Curse of gap. Must do all things myself. ♋ Instincts, form recognition, -why.

P: I tried to make a generic procedure for project value without cash flow. No books help with this. Users reject VVP which I see is enterprise MVP. I refer to potential use of GIS to flipping cost routes. I think CEO is the same as project manager. The owner is the one stakeholder in his own company. ♌♏

B: Why HR software not using API to determine leadership skills. Why do they rely on explicit mentioned “skills” on biography? Labels is meaningless. Gunning Fog index can at least determine “communicating vision”. ♋ ♒

O: Astrology is observation. No experiments can be used. Theoretical physics is equal to astrology. The formulaic statement requirement is gone. Yet Marco could not see the similarities. Both use Sun cycles to determine market. ♉

V: I have no money to AI setup. Therefore I will ghost my business venture. Without my diverse interests marketers would say “I am in x. We are y to z.” instead of “O is 4x cheaper than Q.” despite one more parameter exist. ♑♈

Spectral clustering is the closest alternative to MIR. 1) Matrix to trigrams from row, column, diagonal. 2) Combinatorics from list of tuples. 3) Sample a representative subset of orderings and average the integrals, 3b) Line Sum or Least Square (LSM). 4) RMS of new list, 5) weighted score to labels from 3 relations. Alternatives: {MI} Euclidean, dynamic time warping (DTW) for time series (EMA). {IR} kernel k-means, spectral*, hierarchical, or density-based clustering (DBSCAN) to group data points based on their trigram similarity. {M} PCA, eigenvalue, dim-reduce.

{I} During the clustering process, when assigning a data point to a cluster, consider not just its similarity to the cluster centroid (average of points in the cluster) but also the "sum" of its similarities to other data points within the cluster. This could be achieved by calcul a weighted average of its similarities to other cluster members.

I give people the resources they might need or empowerment and timeline for how they would do it efficiently. YouTube is enough for ruling example by its recognition, relatable, granularity. People Man → Operations Man → CISCO → Head → Prince. System, Consultants (IPsoft), Bankers, Industry, Government, Technology (Atea). Marketing Manager, Product Manager, VP, Presidents, Others, Executives. (B6).

1. Driving: availability heuristic.
2. Toolcard: picture superiority
3. Sumatosoft: cheerleader.
4. Pip Decks: mental accounting
5. Hemingway: post-purchase.
6. Fintech: cross-race, moral luck
7. Blind: implicit, delmore.
8. Leads: backfire, bandwagon
9. Linus: anecdotal.
10. Lazyapply: just-world, fixed
11. Saving: restraint, hyperbolic.
12. Namecheap: time-saving
13. Quora: transparency, google, leveling.
14. Debt: risk-effort
15. Osint: cpi-unit, rosy retrospection.
16. API: loss aversion
17. Projekt: framing, endowment, ego.
18. *Elasticloading: actor, extern agent
19. Ping: hard-easy.
20. Upsun: outcome, subaddivity
21. *Arduino: next-in-line, decoy.
22. PhD-Neptune: expectation
23. Outscraper: suffix, modality.
24. Preferences: essentialism
25. *Chunking: spacing, cryptomensia.
26. Matrix: source, mere exposure
27. Pickle: asymmetric insight.
28. Frequency: clustering, base rate
29. Paradox: conjunction.
30. *Flask: duration neglect, peak-end

---

1. Define introspective intent. 2. Cannot consider nested groups. 3. Reduce social phenomena to concepts such as mutual belief. 4. Require special conventions to be held. 5. See motives as exclusively numbers. [4] Puzzles. [3] 15 workflows. [2] No calculus or Pip Decks. [1] osint (aws.xls) behind imgo.

Stickman RPG. Farm: Hay Day 2,7 Farm Heroes 1,3 Family Farm 3 FarmVille 3 Blossom Blast Arkad: Surfers 1,4 Candy Crus 3 Geometry Dash Action 18,4 Farm. 10 Arcade 4,2 : Farm 3, Ark 2 W arcade Family Farm. W farm Blast or Stickman B arcade unknown. B farm FarmVille. HAU, LA=HF. The lowest arcade is the highest farmhouse. Chapter Entrepreneurship: Auditor. Surveys. Memoization array in JS.

Job interviews for IT usually don't have puzzle but these three may appear: 1) You have 2 supposedly unbreakable light bulbs and a 100 story building. Use the smallest drops possible to determine how much impact this type of bulb can withstand. (ie it can withstand a fall from the 17th floor, but breaks from the 18th). Note that the ever popular binary search gives you a worst case of 50 drops. You should be able to do it with under 20. 2) There are n gas stations along a circular road. Each has a limited supply of gas. You can only drive clockwise around the road. You start with zero gas. Knowing how much gas you need to get from each gas station to the next and how much gas you can get at each station, design an algorithm to find the station you need to start at to get all the way around the circle. 3) Out of 10 coins, one weighs less than the others. You have a wave. How can you determine which weighs less in 3 weights? Now how would you do that if you didn't know if the odd coin weighs less or more?

I help {Niche} to {Offer + end result} in {Offer timeframe} without {Common pain in the niche} using {Mechanism}. “If we don’t get results X in time Y, the whole setup fee gets refunded”. “If during the first 90-days, you do not like the way we are working or us as people, we will refund the full setup fee”. You can make an offer where you are working on a full revenue share model but apart from the revenue share you also implement a testimonial as in your offer.

Monitor	Design	CRUD	Optimize	Debug	Coding
Honey glue-	Dish soup+	Vinegar/	Drown	Drops*	Pesticide
Tension-	Turning*	Temptation/	Timeline+	Teach	Slack
Problem/		Practical+	Promise 3xQ		Pitch
Observer+	Beneficiary*	Victim-		Perpetrator/	Role
Hiring-		Area*		Supply+	Jobs
Lingo24/	Lazyapply-	Grunenthal+		Sport*	Degree
GAI/	QRI+	NVI-	CPI*	AWM	WAIS
1/22 follow	1/10 vote	1/12 deal	5:1 view/vote	1.32 daily	Quora
1/3360 client	1/21 reply	1/32 booking	840 or 16xQ	0.35 daily	Instantly
(/) NOR		(+) AND	(-) NAND	(*) XOR	Modality
by, of, in		to, for, while	or, from, but	is, use	Connective
3.2, 3.4, 4.3	2.1, 2.3, 1.2	5.4, 5.6, 6.5	9.8, 1.9, 9.1	7.8, 8.7, 8.9	Enneagram
3,18,22,4,20,8	13,28,15,30,16	10,19,25,5,9,23	1,14,21,2,17,6	12,27,26,7,24,11	Biases
Manage	Motivating	Planning	Discipline	Speaking	Leadership
TD-RL	DDPG	MCTS	BFGS	Backtrack	Mechanism
ChatSpot	Pictory	Jasper	Lavender	Fireflies	Chatbots
Knowledge	Identity	Mindful	Memory	Transform	Aristotle
Limits	Plurality	Subsistence	Cause	Reciprocal	Kantian

If they specifically ask for the video I would send them the video link but if they're asking for more info I would get them on a call asap. Here's a quick video breakdown: ((link)) Btw — are you free anytime this week? I'd love to give you a trial. If you're down — How's ((day)) working for you? B2B: The only things we are going to change are most likely just going to be messaging and the offer.

14-21 d launch. Hemingwayapp, thereadtime, lucid, notion, clutch, kapwing, dall-e 2. Answerthepublic, urltrends, bustaname, domainling. Verama, OnSiter. Apify, BuzzSumo. Outscraper, ListKit. D1 interview call, linkedin. D2 vsl, website, gurantee. D2 leads, slack, follow-up. D3 campaign, team, millionverifier, airtable. D4 podcast, crm. BBB complain. **1st campaign 8 bd meeting 4 bd client (tis-tor)**

Reply: 45 (9-15) * 74 = 3:19 t/d * 5 = 4 leads. Hypercube: 45a, 74b, 57c, 50d where a is variation, b m, c h, dicp. 4p board=hours, greed=mean. 1d row. 2d col. 3d piece. 4d value. 5d allies. 6d enimes. 7d flag. 8d capture. 9d loss. 10d center. 11d corner. 12d blockade. 13d pincer. 14d fork. 15d hidden. 16d surprise. 4p: roles (3-6), board (1-2, 10-11), greed (7-9), tactics (12-16). Leverage endings / call to actions through lead magnets (VSL. Free Samples. Free Cold Email System Deployment. Free Trial. Podcast/interview. Free Software. Coupons/Gift Cards. ROI Calculators. ICP Calculators. Personalized Blueprints / Roadmaps).

1. Niche Headline (learnings or new case studies). 2. Framing Solution / Evidence (learnings or new case studies). 3. Who This Is For. 4. Core Concept (your new / optimized system & learnings). 5. Credibility (case studies or own campaign results). 6. Thesis (your new / optimized system). 7. Thesis Summary. 8. Option 1 and Option 2. 9. Features/How It Works. 10. Offer/Call To Action. 11. Bonus. 12. Guarantee/Risk Reversal. 13. Summary/Timeline.

requirements	Takes the given requirements and codes to spec	Come up with questions regarding missed cases in the spec	Understand complete picture and come up with entire areas that need to be speced	Able to suggest better alternatives and flows to given requirements based on experience	scripting	No knowledge of scripting tools	Batch files/shell scripts	Perl/ Python/ Ruby/ VBScript/ Powershell	Has written and published reusable code	database	Thinks that Excel is a database	Knows basic database concepts, normalization, ACID, transactions and can write simple selects	Able to design good and normalized database schemas. Knows difference between clustered and non-clustered indexes. Proficient in use of ORM tools.	Can do basic database administration, able to replace cursor usage with relational sql, understands how databases can be mirrored, replicated etc. Understands how the two phase commit works.
languages with professional experience	Imperative or Object Oriented	Imperative, Object-Oriented and declarative (SQL), added bonus if they understand static vs dynamic typing, weak vs strong typing and static inferred types	Functional, added bonus if they understand lazy evaluation, currying, continuations	Concurrent (Erlang, Oz) and Logic (Prolog)	domain knowledge	No knowledge of the domain	Has worked on at least one product in the domain.	Has worked on multiple products in the same domain.	Domain expert. Has designed and implemented several products/solutions in the domain. Well versed with standard terms, protocols used in the domain.	tool knowledge	Limited to primary IDE (VS.Net, Eclipse etc.)	Knows about some alternatives to popular and standard tools.	Editors, debuggers, IDEs, open source alternatives etc. etc. For e.g, someone who knows most of the tools from Scott Hanselman's power tools list. Has used ORM tools.	Has actually written tools and scripts, added bonus if they've been published.
data structures	Doesn't know the difference between Array and LinkedList	Able to explain and use Arrays, LinkedLists, Dictionaries etc in practical programming tasks	Knows space and time tradeoffs of the basic data structures, Arrays vs LinkedLists. Able to explain how hashtables can be implemented and can handle collisions, Priority queues and ways to implement them etc.	Knowledge of advanced data structures like B-trees, binomial and fibonacci heaps, AVL/Red Black trees, Splay Trees, Skip Lists, tries etc.	algorithms	Unable to find the average of numbers in an array (It's hard to believe but I've interviewed such candidates)	Basic sorting, searching and data structure traversal and retrieval algorithms	Tree, Graph, simple greedy and divide and conquer algorithms, is able to understand the relevance of the levels of this matrix.	Able to recognize and code dynamic programming solutions, good knowledge of graph algorithms, good knowledge of numerical computation algorithms, able to identify NP problems etc.	systems programming	Doesn't know what a compiler, linker or interpreter is	Basic understanding of compilers, linker and interpreters. Understands what assembly code is and how things work at the hardware level. Some knowledge of virtual memory and paging.	Understands kernel mode vs. user mode, multi-threading, synchronization primitives and how they're implemented, able to read assembly code. Understands how networks work, understanding of network protocols and socket level programming.	Understands the entire programming stack, hardware (CPU + Memory), binary code, assembly, static and dynamic linking, compilation, interpretation, JIT compilation, garbage collection, heap, stack, memory addressing…
source code version control	Folder backups by date	VSS and beginning CVS/SVN user	Proficient in using CVS and SVN features. Knows how to branch and merge, use patches setup repository properties etc.	Knowledge of distributed VCS systems. Has tried out Bzr/Mercurial/ Darcs/Git	build automation	Only knows how to build from IDE	Knows how to build the system from the command line	Can setup a script to build the basic system	Can setup a script to build the system and also documentation, installers, generate release notes and tag the code in source control	automated testing	Thinks that all testing is the job of the tester	Has written automated unit tests and comes up with good unit test cases for the code that is being written	Has written code in TDD manner	Understands and is able to setup automated functional, load/performance and UI tests
problem decomposition	Only straight line code with copy paste for reuse	Able to break up problem into multiple functions	Able to come up with reusable functions/objects that solve the overall problem	Use of appropriate data structures and algorithms and comes up with generic/object-oriented code that encapsulate aspects of the problem that are subject to change.	systems decomposition	Not able to think above the level of a single file/class	Able to break up problem space and design solution as long as it is within the same platform/technology	Able to design systems that span multiple technologies/platforms.	Able to visualize and design complex systems. Also should be able to design operations support systems like monitoring, reporting, fail overs etc.	communication	Cannot express thoughts/ideas to peers. Poor spelling and grammar.	Peers can understand what is being said. Good spelling and grammar.	Is able to effectively communicate with peers	Able to understand and communicate thoughts/design/ideas/specs in a unambiguous manner and adjusts communication as per the context
code organization within a file	no evidence of organization within a file	Methods are grouped logically or by accessibility	Code is grouped into regions and well commented with references to other source files	File has license header, summary, well commented, consistent white space usage. The file should look beautiful.	code organization across files	No thought given to organizing code across files	Related files are grouped into a folder	Each physical file has a unique purpose, for e.g, one class definition, one feature implementation etc.	Code organization at a physical level closely matches design and looking at file names and folder distribution provides insights into design	source tree organization	Everything in one folder	Basic separation of code into logical folders.	No circular dependencies, binaries, libs, docs, builds, third-party code all organized into appropriate folders	Physical layout of source tree matches logical hierarchy and organization. The directory names and organization provide insights into the design of the system.

	ADVANTAGES	DISADVANTAGES
(Golang)	+ Fast execution speed + Minimal runtime errors due to static typing + Concurrent + Cross-platform + Built-in garbage collection + Single executable	- Less flexible due to static typing - Limited developer familiarity
Python	+ Modular + Flexibility due to dynamic typing + Object-oriented + Extensive third-party libraries and modules	- Slow execution speed - Runtime errors due to dynamic typing - Lower memory efficiency - Potentially complicated runtime environment setup
Scala	+ Similar syntax to Java + Support for highly scalable/concurrent systems + Immutable objects + First-class functions	- Runs in the JVM, which can cause restraints, depending on runtime environment - Limited developer familiarity - Smaller community
Ruby	+ Large and active community + Extensible with many third-party modules + Built-in garbage collection	- Lower performance than compiled languages, notably in boot and execution speed - Documentation less widely available - Tight Active Record coupling
C&C++	+ Fast execution speed + Extensive developer base and documentation + Extensive feature set, depending on language choice	- No garbage collection - Long build times - Different compiler requirements per OS - ABI incompatibility due to compiler requirements

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand RHEL base pricing: 0.0708 USD per Hour

On-Demand SUSE base pricing: 0.0108 USD per Hour

On-Demand Linux base pricing: 0.0108 USD per Hour

On-Demand Windows base pricing: 0.02 USD per Hour

t3.small

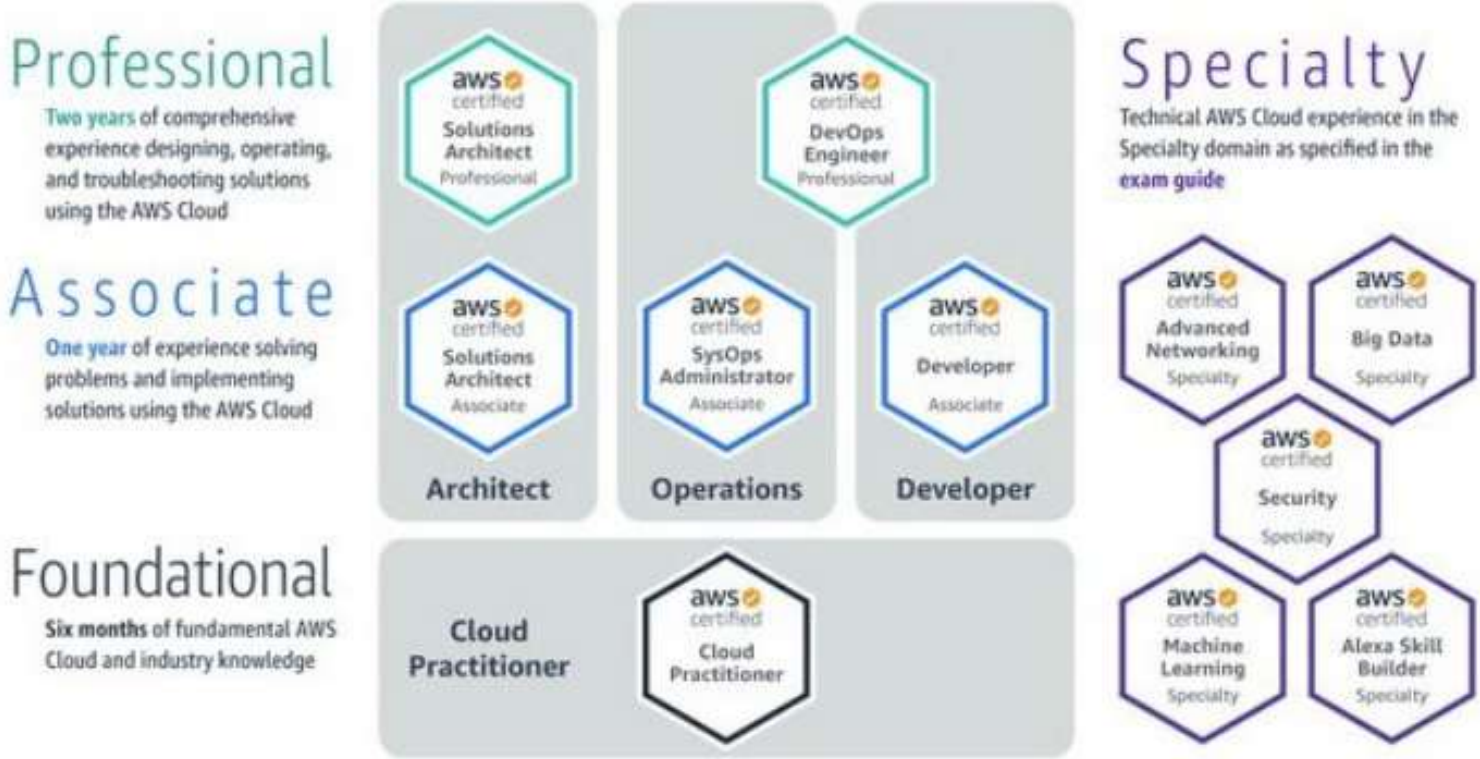
Family: t3 2 vCPU 2 GiB Memory Current generation: true

On-Demand SUSE base pricing: 0.0526 USD per Hour

On-Demand Linux base pricing: 0.0216 USD per Hour

On-Demand RHEL base pricing: 0.0816 USD per Hour

On-Demand Windows base pricing: 0.04 USD per Hour



Aurora Zero-ETL with Redshift. Real-time and ML on transactions without building pipelines.

Company	Experts in	Company size	Price range	Tech stack	Headquarters	Top service
Brainhub	fintech apps, wealth management	100+	$50 - $99 / hr	JavaScript, .NET	Poland	custom software development
10 Clouds	banking systems, accounts integration	200+	$50 - $99 / hr	JavaScript, Python	Poland	AI development
Geniusee	digital banking, blockchain	100+	$25 - $49 / hr	Django, JavaScript	Poland	custom software development
Cheesecake Labs	blockchain, payment systems	100+	$50 - $99 / hr	JavaScript, Python	Brazil	mobile app development
Future Processing	blockchain, banking apps	1000+	$50 - $99 / hr	.NET, Java	Poland	mobile app development
Monterail	finance management apps	150+	$50 - $99/ hr	Ruby, JavaScript	Poland	web development
Miquido	in-app bank account management	200+	$50 - $99 / hr	JavaScript, Golang	Poland	mobile app development
Kind Geek	finance management	100+	$50 - $99 / hr	JavaScript, Java	Poland	mobile app development
Diceus	banking solutions, online payments	100+	$50 - $99 / hr	JavaScript, Golang	Ukraine	cloud consulting
S-Pro	fintech apps, risk management	250+	$50 - $99 / hr	JavaScript, Python	Ukraine	web development
Netguru	banking platforms, investment platforms	900+	$50 - $99 / hr	Ruby, JavaScript	Poland	mobile app development
Dataart	trading systems, digital banking	5500+	$50 - $99 / hr	.NET, JavaScript	USA	custom software development

I saw you in a mutual LinkedIn group and out of curiosity looked a bit through Fondateur Rconseils.

I wondered if you guys are looking for (OFFER) at this moment?

Unsolicited vendor e-mails with a phone call five minutes later will also get your number blocked. Please don't do that.

Honestly, though, you might pique my interest with a subject line like "Let's go get a beer at (Insert local watering hole here)". I still might not respond, but at least you got my attention. If I remember your company name, I might call you back later if and when I actually need your product.

A common point many in this sub will want is simple prices available without requiring a discussion first. People here are often 2-5 man IT depts, some are the sole IT person with budgets that reflect that.

Tell me it's $100/user means it can be ballparked or written off as out of reach without wasting hours of everyone's time.

You screwed up in the first sentence. You want to know how to sell to techs? Don't start with "this is my product/offering/solution." Start with: "Hey I'm in sales and relatively new at it. I don't know you. I don't know your problems, but I'd love to buy you a coffee or something and learn more. Call me if you are interested. (Contact info)"

Announcing your product family is handing the tech an objection. (At least one.) They won't read further unless they need that problem solved right now.

Put in the time and create the relationship. Let the problems and referrals come to you.

Also. The "I love technology! I do home stuff! I'm one of you! Pick-me BS." Demanding street cred never works. Your best bet is to keep silent on that score and just demonstrate competence.

- **GetResponse.** This tool helps manage email subscribers.
- **G Suite.** This is Google's project management app that lets users manage all day-to-day tasks, such as meetings, emails, calendars, presentations, and storing and sharing files.
- **Calendly.** This software helps schedule appointments and sends reminders directly to clients.
- **WorkflowMax.** This project management tool handles all the CRM duties, such as invoices and purchase orders.
- **Expensify.** This software helps businesses track expenses and keep personal and business transactions separate. It also integrates with other major platforms, such as QuickBooks, Sage and Xero, for tax time.

Stratego AI exploit more than what it explore. Unlike AlphaGo it must explore. Rule of third for choosing audience. 1) Not Sweden because of no service like D2D. 2) Not USA because of credit card bias. It has Germany-like restrictions. 3) Not makerspace or sharktank because of beacon and tty3. 4) Not inventor because of non-disclosure. 5) Not mobile games because of value alignment. How to choose niche for RST. 1) Target writing. 2) Diplomacy. 3) Reply behavior. 4) Similar products: Amelia and Zvisofer. 5) Health score for another peer. Small business pros. 1) EU loans. 2) Factoring. 3) Banking.

Dove (psf, soda, storm, warm, oliver). Raven (autogpt, flutter, hackathon, yml). Dove: brute-force with email / domain setup. Raven: undo this, begin with vsl instead. App: cv/ocr, qr/note, smart contract, autogpt, 47. Web: pdf-plugin, contacts, memory, 39.

2018-2023 Hunger Games : Acquiring skills. Building foundation. Taking risks.
2024-2029 Starcraft : Refining skills. Specific targets. Automation.

android-guidelines, klepbot mobile, qr, invoice reminder, flutter-github-workflow, flutter starter, doc-llm, sui, autogpt, hackathon, flutter-docker. Write + VSL > Domains + Campaign. Similtude (hackathon). Expertise: avoiding shinking ships (flashback).

2.6 Lead Gen-formulär på sidor. Om du använder sidfunktionen som tillåter LinkedIn-medlemmar att skicka sin kontaktinformation och annan information till dig ("**Lead Gen-formulär på sidor**"), måste du inkludera en länk till din integritetspolicy i varje sådant formulär. Du får endast använda uppgifterna som skickas till dig via Lead Gen-formulär på sidor ("**Formulärdata**") för lagliga ändamål som de registrerade LinkedIn-medlemmarna godkänner. Eventuella anpassade frågor i ditt registreringsformulär får inte begära känsliga uppgifter, inklusive data som avslöjar en individs ras eller etniska ursprung, politiska åsikter, religiösa eller filosofiska övertygelser, fackföreningsmedlemskap, uppgifter om hälsa eller sexuell läggning, uppgifter relaterade till brott, statligt utfärdade ID-kort, inloggningar och lösenord, finansiell kontoinformation samt information om minderåriga under 16 år.

LinkedIn har rätt att efter eget omdöme avvisa eventuella anpassade frågor. Generellt kommer Formulärdatan att vara tillgänglig för dig eller en godkänd tredje part via LinkedIn i 90 dagar, och du eller denna tredje part är ansvarig för att direkt skaffa en kopia av och lagra Formulärdatan å dina vägnar.

Med förbehåll för din sekretesspolicy kopplad till Lead Gen-formulär på sidor, rimliga integritetsförväntningar hos registrerade LinkedIn-medlemmar, tillämpliga lagar, dessa villkor och omfattningen av eventuella samtycken som erhållits genom Lead Gen-formulär på sidor, kan du: (a) använda formulärdata och (b) dela endast formulärdata med en tredje part som agerar för din räkning, till exempel din tjänsteleverantör, förutsatt att du är ansvarig för att säkerställa att de följer dessa tjänstevillkor och för att begränsa deras respektive användning av formulärdata för din räkning. Du får inte (i) sälja Formulärdata under några omständigheter eller (ii) använda Formulärdata på ett olagligt sätt (t.ex. olaglig diskriminering). Om någon LinkedIn-medlem kräver att funktionen avaktiveras eller att hans eller hennes formulärsdata ska raderas, ska du respektera ett sådant krav omgående.

Tool	Pricing	Key features
CodiumAI	Free for individual use; $19/user/month for teams	Precise code suggestions, code explanation, automated test generation, code behavior coverage, streamlined collaboration, seamless implementation, multiple language and IDE support
GitHub Copilot	Individual: $10/month per user; Business: $19/month per user	Code suggestions, chat functionality, easy auto-complete navigation, multiple language and IDE support
Tabnine	Free for basic AI code completions; Pro: $12/user/month	Code refactoring assistance, code linting, automatic code documentation, intelligent code completions
MutableAI	Basic: $2/repo/month; Premium: $15/repo/month	AI autocomplete, production quality code, prompt-driven development, test generation, language and IDE integration
Amazon CodeWhisperer	Free for individuals; Professional: $19/month	Code suggestions, function completion, documentation generation, security scanning, language and IDE integration
AskCodi	$9.99/month for membership; Free for personal use	Code generation, answering programming questions, providing code suggestions, IDE Integration
Codiga	Free for individual; Paid subscription starts at $14/month	Static code analysis, code completion, code linting, code refactoring, code reviews, language and IDE support
Replit	Free for individual usage; Hacker plan: $7/month; Pro plan: $20/month	Advanced in-line suggestions, code explanation and comments, mistake detection and correction, interactive learning environment
CodeT5	Free for use as a model	Text-to-Code generation, Code-to-Code translation, code summarization
OpenAI Codex	Free API for everyone to use	Quick setup, AI code completion tool, natural language prompting, supported languages, memory capacity, general-purpose programming model

Momento
Cache

Fast serverless cache supporting key/value, dictionaries, and sets, built for production scale.

Add Integration

Neon
Databases

Fully managed serverless PostgreSQL with modern developer features.

Add Integration

PlanetScale
Databases

MySQL-compatible platform that makes databases infinitely scalable, easier and safer to manage.

Add Integration

Supabase
Databases

The open source Firebase alternative. Full Postgres database, Authentication, Storage, Realtime, and more.

Turso
Databases

Turso is an Edge database that is built on SQLite and runs fast from anywhere in the world.

Upstash Kafka
Messaging

A serverless Kafka offering. Quickly provision a Kafka cluster and connect to using a REST API.

To give you an idea, a $1 million cloud bill can result from a Spark ETL job processing ~1.5Tb per hour 24x7 for 365 days a year. Another example might be an application that receives billions of requests a day from multiple locations in the world. In a large enterprise, there are hundreds of applications at this size—resulting in billion-dollar contracts with cloud-providers. For example, Airbnb had a commitment to spend $1.2 billion on cloud resources over five years at the end of 2019 [3].

At Expedia we slashed costs for a data processing ETL costing $1.1 million dollar a year to a mere $100,000 a year by implementing optimisation practices. That's a 91% cost reduction! Not all companies have applications of such a huge size but imagine cutting your cloud cost by 90% just for a single application or for your entire company.

STEP 1: Challenge your design assumptions. Go and get a list of your most expensive applications and challenge your design assumptions.

Are you building an application that has a 99.999% availability and sub-millisecond latency but realistically users would be good enough with a 99% availability and hundreds of millisecond latency? Are you creating datasets with billions of rows but users would only be using aggregations of some of the measures? Are you landing data real-time but data is only analysed once a day? Are you refreshing the cache every 10 seconds but it's only really changing across days?

All these questions go back to the most important question: how is the application going to be used? What's the business value for it to exist? How is the application helping us to achieve a given goal? Of course, all these answers are very often unclear at the beginning of a project; but that's why design should always be an iterative process—allowing changes to happen as seamlessly as possible. Engineers should embrace evolution and change, aligning application development with impact.

STEP 2: Fine-tune your infrastructure resources to your needs. The second step consists of providing the application with the right resources and tuning it to the right infrastructure.

As an engineer, be aware of how cloud costs are calculated. For example, AWS provides spot instances, where you can bid for the cluster price—this is particularly useful if you have fault-tolerant and flexible applications. Use them if you can—AWS claims up to 90% reduction in costs [4]. Some other considerations you might want to address are:

Are you serving customers globally or only in one geographical area? Do you really need your infrastructure to live across the globe or can you set it up closer to your customer base? Are you over-provisioning your cluster instances? Try to ensure there is enough capacity to handle peak loads without unnecessary costs. Utilise auto-scaling to dynamically adjust resources based on actual demand, preventing overpayment for idle resources. If you're working with data and Spark, make sure you understand Spark concepts and tuning! If you don't, take a look at the following resources [5] [6] [7] [8] [9].

STEP 3: Use AWS Graviton instances. There are little to no drawbacks in utilising AWS Graviton instances. AWS has invested heavily in creating the most cost-effective processors. You can get up to 40% reduction in cloud spending just by switching from an intel-based processor to an ARM-based processor [10]. The only caveat to this is that your application needs to be compatible with the ARM-based processors that Graviton runs on. If you're dealing with a managed service such as RDS or OpenSearch then there's no complication at all in switching—AWS deals with the underlying OS and application

compatibility. If you're building your own application, then you might need to recompile the package depending on which language you're using—Java and other languages require no change whereas Python requires attention.

STEP 4: Monitor your cost spending and educate on cost awareness. Lastly, don't forget to keep monitoring your costs for unexpected peaks and surprises. The cost on day 0 of your application will be different from the cost on day 170. Make sure you keep track of the changes, and you understand why the change is happening: is it stacking s3 storage costs or is it just a one-off spike? Set up the alerts and operational guidebooks!

Importantly, implement cost allocation tags to track spending by department, project, or environment. Avoid the risk of creating a data swamp where cost is untraceable or requires a long journey across different log systems. It should be quick and simple to go back to any given application cost. Final thoughts. Wherever you are working, balancing the delivery of new features with the optimisation of current ones is hard. Who hasn't been pressured to deliver new quirky features at the speed of light. However, it is essential for both engineers and managers to make deliberate and proactive decisions about their current projects, managing risks and opportunities effectively.

Consider using the AWS Lambda PowerTools framework to simulate events and test your Lambda function locally. This can help isolate issues related to input format or unexpected scenarios. Create an SQS queue to act as a buffer for messages between DL and SFW. SQS Poll: Provides a buffer for handling message spikes or delays. Correlation is between latency and memory. I used lambda and codebuild from gh-pages. Here is the benchmark: 78 MB respective 75 seconds. Memory allocation and provisioned concurrency. AWS Edge to increase network latency. Asynchronous workflows like SQS.

Offload as much work as possible to your hosting provider by leveraging managed services. For us, this meant using Kubernetes for our runtime, MySQL for our database, and Redis for caching and streaming. The Dapr project can also help by abstracting your code from the underlying infrastructure, allowing you to bind to more productive Cloud APIs without sacrificing portability. For instance, you can use Dapr's Pub/Sub API to perform asynchronous messaging against Redis or a higher-level service like AWS's SQS with no code change required. Diagrid, Qcon. InfoQ encourages software practitioners and domain experts to submit full-length technical educational articles. Hasura Supergraph.

Documentation: Catchafire and Taproot. Parkinson: gh/kevinjycui/hand-spoken-backend (https://github.com/multunus/first-contributions): This hands-on tutorial offers a step-by-step guide for contributing to open source projects on GitHub. By completing the tutorial, you'll have made your first contribution to the same project.
(https://up-for-grabs.net): This website curates a list of open source projects actively seeking help. Projects are labeled with tags like "up-for-grabs," "jump-in," or "help wanted" to make it easy for potential contributors to identify opportunities to get involved.
(https://goodfirstissues.com): This website aggregates the latest issues from open source projects labeled "Good First Issue" on GitHub. These issues are specifically selected to help first-time contributors get started with open source.
(https://goodfirstissue.dev): This website curates a selection of beginner-friendly issues from popular open source projects. It offers guidance and resources to help you make your first contribution to open source.

Locust: An open-source load testing tool, crucial for simulating traffic and identifying scalability issues in a controlled manner. Imagine a scenario where the traffic to your

Customer Experience vs. User Experience

code readability	Mono-syllable names	Good names for files, variables classes, methods etc.	No long functions, comments explaining unusual code, bug fixes, code assumptions	Code assumptions are verified using asserts, code flows naturally – no deep nesting of conditionals or methods	defensive coding	Doesn't understand the concept	Checks all arguments and asserts critical assumptions in code	Makes sure to check return values and check for exceptions around code that can fail.	Has his own library to help with defensive coding, writes unit tests that simulate faults	error handling	Only codes the happy case	Basic error handling around code that can throw exceptions/generate errors	Ensures that error/exceptions leave program in good state, resources, connections and memory is all cleaned up properly	Codes to detect possible exception before, maintain consistent exception handling strategy in all layers of code
IDE	Mostly uses IDE for text editing	Knows their way around the interface, able to effectively use the IDE using menus.	Knows keyboard shortcuts for most used operations.	Has written custom macros	API	Needs to look up the documentation frequently	Has the most frequently used APIs in memory	Vast and In-depth knowledge of the API	Has written libraries that sit on top of the API to simplify frequently used tasks and to fill in gaps in the API	frameworks	Has not used any framework outside of the core platform	Has heard about but not used the popular frameworks available for the platform.	Has used more than one framework in a professional capacity and is well-versed with the idioms of the frameworks.	Author of framework

languages exposed to	Imperative or Object Oriented	Imperative, Object-Oriented and declarative (SQL), weak vs strong typing and static inferred types	Functional, added bonus if they understand lazy evaluation, currying, continuations	Concurrent (Erlang, Oz) and Logic (Prolog)	codebase knowledge	Has never looked at the codebase	Basic knowledge of the code layout and how to build the system	Good working knowledge of code base, has implemented several bug fixes and maybe some small features.	Has implemented multiple big features in the codebase and can easily visualize the changes required for most features or bug fixes.
knowledge of upcoming technologies	Has not heard of the upcoming technologies	Has heard of upcoming technologies in the field	Has downloaded the alpha preview/CTP/ beta and read some articles/manuals	Has played with the previews and has actually built something with it and as a bonus shared that with everyone else	platform internals	Zero knowledge of platform internals	Has basic knowledge of how the platform works internally	Deep knowledge of platform internals and can visualize how the platform takes the program and converts it into executable code.	Has written tools to enhance or provide information on platform internals. For e.g. disassemblers, decompilers, debuggers etc.

B: Persons are sensitive and a bit of an introvert, but still manage to be personable and compassionate.
R: Persons feel things strongly and their rich, intense inner life emanates outward.
A: Persons are their own person: ambitious and freethinking.
G: Persons are an intellectually active person with loads of drive and the ability to make things happen.
E: Persons are freedom-loving, sensual and enthusiastic.
O: Persons know where the moral high ground is, and always try to take it.
N: Persons are a think outside the box' kind of person -- creative and original.

D

Daniel Å. Jonsson

Jivanmukta with endless energy. ·

Updated May 3

Why is society prone to creating a "Complexity Bias"?

Code to tangent. Ln(H/mol)/Pa is mass energy from gas (false) to liquid (true). It's a preference for memory recall of different topics. "I always lie" have value of 1.19 rest which would be Pa. You thought your vision would be spy glasses with ML engineering / text augmentation. It's a shame that it was not more than SaaS or Excel plugin at worst.

Or waste money to tech stack of sales outreach vs arbitrage betting. Or buy market research report on LinkedIn and create one similar in the future. By losing early as possible you can win at later stages. The real catch for complexity is not being so hopeful for future gains. In software making would Flake8 be seen as vitamin or